Labo 07 — Labo pratique : web → api → db par leurs noms

Objectif : voir ce que le réseau par défaut rootless donne vraiment, créer un réseau avec DNS, publier des ports, assembler la chaîne nginx → API → PostgreSQL par les noms, casser le DNS de nginx pour comprendre le `502`, puis refaire la même chose dans un pod.

Prérequis — Labos 01 à 06 terminés. `~/labo-docker/05/Dockerfile` et `Api.java` présents (labo 05). Images `alpine`, `nginx:alpine`, `eclipse-temurin:21-jdk`, `eclipse-temurin:21-jre-alpine` ; `postgres:16-alpine` sera tirée de nouveau.

Fichiers fournis (`files/`) - `nginx-proxy.conf` — nginx sert le front et relaie `/api/` vers `http://api:8080/`. - `nginx-pod.conf` — la même chose vers `http://localhost:8080/`, pour l'étape pod. - `index.html` — la page « Angular » qui appelle `/api/actuator/health`.

Les adresses `10.89.0.x` ci-dessous sont celles d'un réseau fraîchement créé ; les vôtres peuvent différer d'une unité ou deux.

Étape 0 — Reconstruire l'API

```
mkdir -p ~/labo-docker/07 && cd ~/labo-docker/07
cp <chemin-du-labo>/files/* .
podman build -q -t api:1.0 ~/labo-docker/05
podman images --format '{{.Repository}}:{{.Tag}} {{.Size}}' | grep api
```

Observez `localhost/api:1.0 209 MB` : le multi-stage du labo 05, reconstruit en quelques secondes grâce au cache.

Étape 1 — Le réseau par défaut : pasta

```
podman run -d --name solo alpine sleep 1d
podman inspect solo --format '{{.NetworkSettings.IPAddress}}|{{.HostConfig.NetworkMode}}'
podman exec solo ip -4 -o addr show eth0
ip -4 -o addr show scope global | head -1
podman exec solo cat /etc/resolv.conf
podman exec solo wget -qO- --timeout=5 http://1.1.1.1 | head -c 60 ; echo
podman rm -f -t 0 solo
```

Observez `|pasta` — pas d'adresse, mode `pasta` —, puis une `eth0` dans le conteneur qui porte **la même adresse que votre WSL** (`172.2x.x.x/20`), le `resolv.conf` de l'hôte, et du HTML : Internet fonctionne.

Explication. Sans droits root, pas de bridge : `pasta` reproduit la pile réseau de l'hôte dans le conteneur. Pratique pour un conteneur seul ; inutilisable pour en faire dialoguer plusieurs.

Étape 2 — Le bridge `podman` : des IP, pas de noms

```
podman network ls
podman network inspect podman | grep -E '"(subnet|gateway|dns_enabled)"'
podman run -d --name p1 --network podman alpine sleep 1d
podman run -d --name p2 --network podman alpine sleep 1d
podman inspect p1 --format '{{.NetworkSettings.Networks.podman.IPAddress}}'
podman exec p2 ping -c1 -W1 $(podman inspect p1 --format
'{{.NetworkSettings.Networks.podman.IPAddress}}') | head -2
podman exec p2 ping -c1 -W1 p1
podman rm -f -t 0 p1 p2
```

Observez `"subnet": "10.88.0.0/16"`, `"dns_enabled": false` ; `p1` a une adresse `10.88.0.x` ; `ping` par IP répond (`64 bytes from 10.88.0.x`) ; `ping p1` échoue : `ping: bad address 'p1'`.

Explication. Le bridge relie les conteneurs, mais le réseau `podman` n'a pas de DNS. Coder `10.88.0.4` dans une configuration serait une bombe à retardement : l'adresse change à la prochaine création.

Étape 3 — Un réseau user-defined avec DNS

```
podman network create stack
podman network inspect stack | grep -E '"(name|subnet|gateway|dns_enabled)"'
podman run -d --name db --network stack -e POSTGRES_PASSWORD=secret postgres:16-alpine
podman run -d --name api --network stack -e APP_MESSAGE="bonjour du reseau stack" api:1.0
sleep 3 ; podman exec api cat /etc/resolv.conf
podman run --rm --network stack alpine sh -c 'getent hosts api db; wget -qO-
http://api:8080/actuator/health; echo; nc -zv db 5432'
podman run --rm --network stack -e PGPASSWORD=secret postgres:16-alpine psql -h db -U postgres
-c 'select version()' | head -3
```

Observez `"subnet": "10.89.0.0/24"`, `"dns_enabled": true` ; dans `api`, `search dns.podman` et `nameserver 10.89.0.1` ; puis `10.89.0.3 api.dns.podman … api`, `10.89.0.2 db.dns.podman … db`, `{"status":"UP"}`, `db (10.89.0.2:5432) open`, et `PostgreSQL 16.x on x86_64-pc-linux-musl` : un client jetable a joint la base **par son nom**.

```
podman network inspect stack --format '{{range .Containers}}{{.Name}} {{end}}'
```

Observez `api db` : le réseau connaît ses membres.

Explication. Le réseau créé par vous lance `aardvark-dns` sur la passerelle. Tout conteneur du réseau résout les noms des autres. C'est la seule façon saine de relier des services.

Étape 4 — Publier un port

```
podman image inspect api:1.0 --format '{{.Config.ExposedPorts}}'
podman port api
curl -s --max-time 2 localhost:8080/actuator/health || echo "(injoignable depuis l'hôte)"
```

Observez `map[8080/tcp:{}]` — l'image *déclare* 8080 —, `podman port api` vide, et `(injoignable depuis l'hôte)` : `EXPOSE` ne publie rien.

```
podman run -d --name api-pub --network stack -p 18080:8080 api:1.0
sleep 2 ; podman port api-pub ; curl -s localhost:18080/actuator/health ; echo
podman run -d --name api-rand --network stack -P api:1.0
podman port api-rand
podman run -d --name api-lo --network stack -p 127.0.0.1:18081:8080 api:1.0
podman port api-lo
podman ps --format 'table {{.Names}}\t{{.Ports}}'
```

Observez `8080/tcp -> 0.0.0.0:18080` et `{"status":"UP"}` ; avec `-P`, un port hôte aléatoire (`0.0.0.0:39843`) ; avec `127.0.0.1:`, `127.0.0.1:18081` ; et dans `ps`, `db 5432/tcp` et `api 8080/tcp` **sans flèche** (exposés, pas publiés) contre `api-pub 0.0.0.0:18080->8080/tcp`.

```
podman run -d --name api-80 --network stack -p 80:8080 api:1.0 ; echo "exit=$?"
podman ps -a --filter name=api-80 --format '{{.Status}}'
podman rm -f -t 0 api-pub api-rand api-lo api-80
```

Observez `Error: rootlessport cannot expose privileged port 80, you can add 'net.ipv4.ip_unprivileged_port_start=80' to /etc/sysctl.conf (currently 1024), or choose a larger port number (>= 1024): listen tcp 0.0.0.0:80: bind: permission denied`, `exit=126`, et un conteneur resté `Created`.

Explication. Publier, c'est demander à l'hôte d'écouter pour le conteneur. En rootless, l'hôte c'est vous, et vous n'avez pas le port 80.

Étape 5 — La chaîne web → api → db

```
podman run -d --name web --network stack -p 18080:80 \
  -v ./nginx-proxy.conf:/etc/nginx/conf.d/default.conf:ro \
  -v ./index.html:/usr/share/nginx/html/index.html:ro nginx:alpine
sleep 1 ; curl -s localhost:18080/ | head -3
curl -s localhost:18080/api/actuator/health ; echo
curl -s localhost:18080/api/ ; echo
podman logs web 2>&1 | tail -1
```

Observez le `<!doctype html>` du front, puis `{"status":"UP"}` et `{"message":"bonjour du reseau stack","profile":"default"}` : nginx a relayé `/api/` vers `api:8080` en retirant le préfixe. Dans le log, l'adresse cliente est celle de `web` lui-même (`10.89.0.10` ici : les conteneurs de l'étape 4 ont consommé les adresses précédentes).

Explication. Un seul port publié, celui de nginx. L'API et la base ne sont joignables que par leur nom, depuis le réseau. Ouvrez `http://localhost:18080` dans votre navigateur Windows : la page appelle `/api/actuator/health` et affiche `{"status":"UP"}` — sans CORS, puisque tout vient de la même origine.

Étape 6 — Casser le DNS de nginx

```
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman inspect api --format '{{.NetworkSettings.Networks.stack.IPAddress}}'
podman stop -t 1 api
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman run -d --name intrus --network stack alpine sleep 1d
podman start api ; sleep 2
podman inspect api --format '{{.NetworkSettings.Networks.stack.IPAddress}}'
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman logs web 2>&1 | grep -i error | tail -1 | cut -c1-120
```

Observez `200` et `10.89.0.3` ; après l'arrêt, `504` ; après le redémarrage, l'API a changé d'adresse (`10.89.0.12` : `intrus` a pris la précédente) et nginx répond `502`, avec `connect() failed (113: Host is unreachable) while connecting to upstream`.

```
podman run --rm --network stack alpine wget -qO- http://api:8080/actuator/health ; echo
podman restart -t 1 web ; sleep 1
curl -s -o /dev/null -w '%{http_code}\n' localhost:18080/api/actuator/health
podman rm -f -t 0 intrus
```

Observez que le conteneur jetable joint `api` sans problème (le DNS est à jour), et qu'après `restart` de `web`, `200`.

Explication. nginx fige les IP de ses `proxy_pass` au démarrage. Le DNS n'est pas en cause : c'est le client qui ne le réinterroge pas. Redémarrer nginx, ou lui donner un `resolver`.

Étape 7 — Deux réseaux et un alias

```
podman network create admin
podman run -d --name mon --network admin alpine sleep 1d
podman exec mon wget -qO- --timeout=2 http://api:8080/actuator/health || echo "api injoignable depuis admin"
podman network connect --alias backend admin api
podman exec mon wget -qO- --timeout=2 http://backend:8080/actuator/health ; echo
podman inspect api --format '{{range $k,$v := .NetworkSettings.Networks}}{{$k}}={{$v.IPAddress}} {{end}}'
podman network disconnect admin api
podman rm -f -t 0 mon ; podman network rm admin
```

Observez `wget: bad address 'api:8080'` puis `api injoignable depuis admin` ; après la connexion, `{"status":"UP"}` via l'alias `backend` ; et deux adresses : `admin=10.89.1.3 stack=10.89.0.12`.

Explication. Un réseau est aussi une frontière : `mon` ne voyait pas `api`. Connecter `api` à `admin` lui ajoute une interface et un nom sur ce réseau seulement.

Étape 8 — host, none, et l'hôte depuis un conteneur

```
podman run --rm --network host alpine ip -4 -o addr show | grep -v ' lo '
podman run --rm --network host nginx:alpine sh -c 'nginx; sleep 1' 2>&1 | tail -1
podman run --rm --network none alpine wget -qO- --timeout=2 http://1.1.1.1 2>&1 | tail -1
podman run --rm --network none alpine ip -4 -o addr
podman run --rm alpine sh -c 'getent hosts host.containers.internal; wget -qO-
http://host.containers.internal:18080/api/actuator/health' ; echo
```

Observez vos propres interfaces WSL en `--network host`, puis `nginx: [emerg] bind() to 0.0.0.0:80 failed (13: Permission denied)` ; en `--network none`, `wget: can't connect to remote host (1.1.1.1): Network unreachable` et seulement `lo` ; enfin `169.254.1.2 host.containers.internal` et `{"status":"UP"}` : depuis le réseau par défaut, l'hôte et ses ports publiés sont joignables par ce nom.

Explication. `host` = pas d'isolation (et les limites de votre utilisateur) ; `none` = isolation totale ; `host.containers.internal` = « la machine qui m'héberge », l'équivalent Podman de `host.docker.internal`.

Étape 9 — La même chaîne dans un pod

```
podman pod create --name stackpod -p 18090:80
podman run -d --pod stackpod --name api-pod api:1.0
podman run -d --pod stackpod --name web-pod -v ./nginx-
pod.conf:/etc/nginx/conf.d/default.conf:ro nginx:alpine
sleep 2 ; podman pod ps
podman ps --pod --format 'table {{.Names}}\t{{.Pod}}\t{{.Ports}}'
curl -s localhost:18090/api/actuator/health ; echo
podman exec api-pod cat /etc/hosts | tail -2
podman pod rm -f -t 0 stackpod
```

Observez un pod `Running` avec `3` conteneurs (le troisième est l'*infra*) ; dans `ps --pod`, `api-pod` et `web-pod` partagent le même `POD ID` et la même publication `0.0.0.0:18090->80/tcp`, portée par `…-infra` ; `{"status":"UP"}` via un nginx qui relaie vers `localhost:8080` ; et dans `/etc/hosts`, `127.0.0.1 api-pod` et `127.0.0.1 web-pod`.

Explication. Dans un pod, un seul network namespace pour tous : `localhost` relie les conteneurs, et c'est le pod qui publie. Vous venez d'exécuter, sans le savoir, l'équivalent d'un Pod Kubernetes à deux conteneurs.

Nettoyage

```
podman network rm stack
podman rm -f -t 0 web api db
podman network rm stack
podman network ls
podman rmi docker.io/library/postgres:16-alpine
```

Observez d'abord `Error: "stack" has associated containers with it. Use -f to forcibly delete containers and pods: network is being used`, puis, après la suppression nommée des trois conteneurs, `stack` supprimé et seul `podman` restant. Gardez `api:1.0` et `~/labo-docker/07/` : les labos 08 et 09 réutilisent l'image et les fichiers nginx.

Ce que vous devez pouvoir affirmer maintenant

- Réseau par défaut rootless = `pasta` : `IPAddress` vide, l'adresse de l'hôte copiée, pas de nom.
- Réseau `podman` : des IP `10.88.0.x`, `ping: bad address` par nom ; réseau créé : `dns_enabled: true`, `getent hosts api` répond.
- `EXPOSE` ne publie rien : `podman port api` vide ; `-p 18080:8080` publie ; `-P` tire un port aléatoire ; `-p 80:…` → `permission denied` en rootless.
- nginx relaie `/api/` vers `api:8080` par le nom ; une seule origine, pas de CORS.
- API recréée = nouvelle IP = `502` chez nginx, `200` après `restart web`.
- `--network host` : vos interfaces et vos limites ; `--network none` : `Network unreachable`.
- Un pod partage `localhost` et publie lui-même : `3` conteneurs dont l'infra, `127.0.0.1 api-pod`.